

Creational Design Patterns

Purpose: Creational patterns are focused on object creation mechanisms, aiming to optimize the creation process while ensuring flexibility.

Goal: They abstract the instantiation process to make systems more flexible and reusable.

Problem They Solve: Prevent tight coupling between code and object creation logic, simplifying the management of new object creation, especially in complex systems. Instead of the client creating objects directly (e.g. calling `new` everywhere), an extra layer takes on the responsibility of object creation, and the client simply requests an object from it.

Example: In a system where different types of documents (PDF, Word, Excel) are created, instead of using `new` everywhere, a Factory Pattern can centralize object creation into one place.

Application: Creational patterns are applied in scenarios where: - You want to separate the instantiation process from usage. - The exact type of object needed can vary depending on runtime conditions.

The patterns covered in this module: **Singleton, Builder, Prototype, Factory Method, and Abstract Factory.**

Singleton Pattern

Category: Creational Design Pattern

1. The Problem

Consider an `AppSettings` class that reads configuration (DB URL, API key) from a config file. Without any restriction, any part of the code can create as many instances as it wants.

```
public class AppSettings {

    private String dbUrl;
    private String apiKey;

    public AppSettings(){
        //Read settings from a config file
        dbUrl = "jdbc:mysql://localhost:3306/mydb";
        apiKey = "12345";
    }

    public String getApiKey() {
        return apiKey;
    }

    public String getDbUrl() {
        return dbUrl;
    }
}

public class WithoutSingletonPattern {
    public static void main(String[] args) {
        AppSettings appSettings = new AppSettings();
        AppSettings appSettings1 = new AppSettings();

        System.out.println(appSettings.getApiKey());
        System.out.println(appSettings1.getApiKey());
        //More memory
    }
}
```

Issues:

- **Inconsistent state** — if multiple instances represent the same concept (like app-wide settings), they could end up holding different data.
- **Resource conflicts / waste** — creating multiple instances of a resource-heavy class (e.g. something wrapping a DB connection) wastes memory and can degrade performance.
- No guarantee at all that only one `AppSettings` exists — every `new AppSettings()` call silently creates another, separate object.

2. The Solution — Singleton Pattern

Intent: Ensure a class has exactly one instance throughout the application's lifecycle, and provide a single, well-known global point of access to it.

- The constructor is made `private`, so no other class can instantiate `AppSettings` directly.
- A `public static getInstance()` method controls access to the one instance — creating it the first time it's needed, and returning the same object every time after.

3. Key Idea

- **Control instantiation from within the class itself** — instead of relying on callers to “behave” and only create one instance, the class makes it *impossible* to create more than one by hiding the constructor.
- **Lazy initialization** — the instance is only created the first time `getInstance()` is called, not upfront, saving resources if it’s never needed.
- Any part of the application accesses the same shared object via `AppSettings.getInstance()`, giving a single, consistent source of truth.

4. Variants — Thread Safety

The basic lazy version shown above is **not thread-safe**: in a multi-threaded environment, two threads could both pass the `instance == null` check at the same time and create two separate instances. Common fixes:

- **Lazy Initialization with Synchronization** — mark `getInstance()` as `synchronized`, so only one thread can execute it at a time. Simple, but adds a small performance cost on every call, even after the instance already exists.
- **Double-Checked Locking** — only synchronize the block where the instance is actually created, checking `instance == null` both before and inside the synchronized block. Avoids the overhead of synchronizing every single call.
- **Bill Pugh Singleton Design** — uses a private static inner “holder” class that isn’t loaded until `getInstance()` is first called. The JVM’s class-loading mechanism guarantees thread safety and lazy loading without any explicit synchronization.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Singleton class	Holds a private static reference to its own single instance; exposes a static accessor; hides its constructor	<code>AppSettings</code>
Client	Retrieves the shared instance via the static accessor instead of using <code>new</code>	<code>SingletonPattern</code> (main method)

Relationship type: Not a relationship between two classes in the usual sense — the class manages a relationship *with itself*, holding a single static self-reference (`private static AppSettings instance`).

6. Code — Full Implementation

```
public class AppSettings {

    // Step 1: private static instance of the class
    private static AppSettings instance;

    private String dbUrl;
    private String apiKey;

    // Step 2: Private constructor to prevent direct object creation
    private AppSettings(){
        //Read settings from a config file
        dbUrl = "jdbc:mysql://localhost:3306/mydb";
        apiKey = "12345";
    }

    // Step 3: Public static method to get single instance
    public static AppSettings getInstance(){
        if (instance == null){
            instance = new AppSettings();
        }
        return instance;
    }

    public String getApiKey() {
        return apiKey;
    }

    public String getDbUrl() {
        return dbUrl;
    }
}

public class SingletonPattern {
    public static void main(String[] args) {

        AppSettings appSettings = AppSettings.getInstance();
        AppSettings appSettings1 = AppSettings.getInstance();

        System.out.println(appSettings.getApiKey());
        System.out.println(appSettings1.getApiKey());
        System.out.println(appSettings1 == appSettings);
    }
}
```

What changed vs. the “without pattern” version: - Public constructor → private constructor, so `new AppSettings()` is no longer possible from outside the class. - No instance tracking → a private static `AppSettings instance` field holds the one-and-only object. - Client called `new AppSettings()` directly → client now calls `AppSettings.getInstance()`, which creates the instance only once and reuses it afterward. - `appSettings1 == appSettings` now prints `true` — both references point to the exact same object.

7. Benefits

- **Controlled access to a single instance** — guarantees exactly one object exists for the class's lifetime.
- **Reduced memory footprint** — avoids creating redundant instances of resource-heavy objects (e.g. DB connections, config data).
- **Global access point** — any part of the application can reach the same shared instance consistently.
- **Lazy initialization** — the instance is only created when actually needed, not upfront.

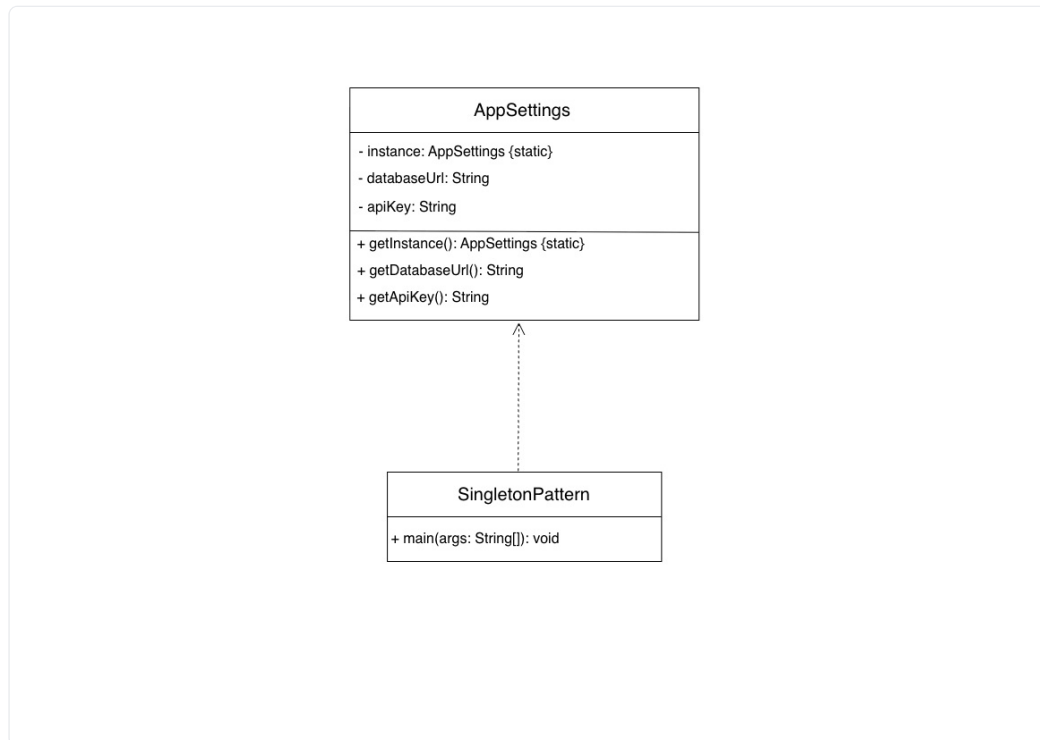
8. Drawbacks / Things to Watch Out For

- **Thread-safety concerns** — the naive lazy implementation can create multiple instances under concurrent access unless properly synchronized.
- **Hidden dependency** — code that calls `getInstance()` anywhere in the codebase creates an implicit global dependency, which can make unit testing harder (mocking a singleton is trickier than mocking an injected dependency).
- **Can become a “God object”** — over time, a singleton can accumulate too many responsibilities since it's so easy to reach from anywhere.
- Overuse of Singleton is sometimes considered an anti-pattern when it's used as a substitute for proper dependency injection.

9. Real-World Use Cases

- **Database connection management** — a single connection pool/manager shared across the app.
- **Logging** — a single logger instance used app-wide to keep log output consistent and centralized.
- **Configuration settings** — application-wide settings (like `AppSettings` here) that should be read once and shared everywhere.
- **Caching** — a single shared cache instance to avoid duplicate/conflicting caches.

Diagram



Singleton Pattern UML diagram

`AppSettings` holds a private static field `instance` of its own type, along with instance data (`databaseUrl` , `apiKey`). It exposes a static `getInstance()` method as the only way to obtain the object, plus regular getters. `SingletonPattern` (the client) depends on `AppSettings` only through `getInstance()` — it never constructs `AppSettings` directly.

10. Interview Questions & Answers

Q1. What is the Singleton Pattern? A creational design pattern that ensures a class has exactly one instance throughout the application's lifecycle, and provides a single global point of access to it.

Q2. What problem does it solve? It prevents inconsistent state and resource waste that can occur when multiple instances of a class meant to represent one shared concept (like configuration or a connection manager) are accidentally created.

Q3. What are the key implementation elements of a Singleton? A `private` constructor to block direct instantiation, a `private static` field holding the single instance, and a `public static` accessor method (`getInstance()`) that creates the instance on first use and returns it on every subsequent call.

Q4. Why is the naive Singleton implementation not thread-safe, and how do you fix it? Two threads can both evaluate `instance == null` as `true` at the same time before either has finished creating the object, resulting in two separate instances. Fixes include synchronizing `getInstance()` , using double-checked locking to synchronize only the creation block, or using the Bill Pugh (static inner holder class) approach, which relies on the JVM's class-loading guarantees instead of explicit locks.

Q5. What is Double-Checked Locking, and why is it preferred over simply synchronizing the whole method? It checks `instance == null` once before entering a synchronized block, and again inside it before creating the instance. This way, synchronization overhead is only paid the first time (during creation), not on every subsequent call, since after the instance exists the outer null-check short-circuits before reaching the synchronized block.

Q6. What is the Bill Pugh Singleton design, and why is it considered a best practice? It uses a private static inner “holder” class containing the singleton instance. The JVM only loads (and initializes) that inner class the first time it’s referenced — i.e. the first call to `getInstance()` — giving lazy initialization and thread safety for free, without needing any explicit `synchronized` keyword.

Q7. What are the benefits of the Singleton Pattern? Guaranteed single instance, reduced memory usage for resource-heavy objects, a consistent global access point, and lazy initialization.

Q8. What are the drawbacks or criticisms of the Singleton Pattern? Thread-safety complexity if not implemented carefully, hidden global dependencies that complicate unit testing, the risk of the singleton growing into an overloaded “God object,” and general overuse as a substitute for proper dependency injection.

Q9. Can you give real-world examples? Database connection managers, application-wide loggers, shared configuration objects, and shared cache instances.

Q10. What’s the difference between Eager and Lazy initialization of a Singleton? Eager initialization creates the instance upfront, as soon as the class is loaded (e.g. `private static AppSettings instance = new AppSettings();`) — simple and inherently thread-safe, but wastes resources if the instance is never actually used. **Lazy initialization** (as in this example) defers creation until `getInstance()` is first called, saving resources but requiring extra care for thread safety.

Q11. Can a Singleton be broken — i.e. can more than one instance ever get created? Yes, a few common ways: reflection can invoke a private constructor directly; Java serialization/deserialization can produce a new object unless `readResolve()` is implemented to return the existing instance; and cloning can bypass the constructor entirely unless `clone()` is overridden to return the singleton itself.

Q12. How would you test code that depends on a Singleton? Since a Singleton is a global, hardcoded dependency, it can’t easily be swapped for a mock/test double the way an injected dependency can. Common approaches include exposing an interface for the Singleton’s behavior and injecting it (so tests can substitute a fake), or providing a package-private “reset” method used only in test setup — though many consider heavy Singleton use a sign that dependency injection would serve the design better.

Q13. Is Singleton considered an anti-pattern? Why or why not? It’s not inherently an anti-pattern — it solves a real problem (guaranteeing one instance) cleanly. It gets criticized when it’s overused as a convenient global variable substitute, hiding dependencies that would be clearer and more testable if passed in explicitly via constructor/dependency injection instead.